Name _____ SID _____

CS/EE 147
GPU Architecture and Parallel Programming

Spring 2026

Practice Exam 2

Time Allowed: 80 Minutes

Question	Points Possible	Points Scored
1	4	
2	4	
3	4	
4	13	
5	12	
6–9 (MC)	12	
10–15 (T/F)	12	
16	6	
17	12	
18	4	
Total	83	

- Closed book, closed internet
- Allowed 1 page letter-size cheatsheet, front and back
- Calculators allowed, but likely not necessary
- Covers Lectures 8–14: Unified Memory, Matrix Multiply, Memory Coalescing, Histogram, Multi-GPU, GPU Sharing (slides 1–25), Dynamic Parallelism

1. **[4 points] Tiled SGEMM Launch**
   You launch a tiled matrix-multiply kernel on a 1024×1024 output matrix with `TILE_WIDTH = 32`.

   a. [2 points] How many warps are in each block?

   Answer: [ ]

   b. [2 points] How many blocks are in the grid?

   Answer: [ ]

2. **[4 points]** Below is the loop body of an interleaved-partition histogram kernel. What should `stride` be initialized to so that all threads in the grid jointly cover the input with a coalesced access pattern?

```
__global__ void histo_kernel(unsigned char *buf, long size,
                             unsigned int *histo) {
    int i = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = ????
    while (i < size) {
        int pos = buf[i] - 'a';
        if (pos >= 0 && pos < 26)
            atomicAdd(&histo[pos/4], 1);
        i += stride;
    }
}
```

`int stride =` [ ]

3. **[4 points]** A teammate proposes that in the kernel above, having every block maintain a `__shared__` private histogram and merging into the global histogram at the end will significantly improve performance. Briefly explain *why* this typically helps.

Answer: [ ]

4. **[13 points] Unified Memory**
   You write the following kernel and host program on a **Pascal-class** GPU (CUDA 8+):

```
__global__ void mykernel(char *data) {
    data[1] = 'g';
}

void foo() {
    char *data;
    cudaMallocManaged(&data, 2);
    mykernel<<<1,1>>>(data);
    // no synchronize here
    data[0] = 'c';
    cudaFree(data);
}
```

a. [3 points] On Pascal+, what happens when the CPU executes `data[0] = 'c'` while the kernel may still be running?

Answer:

b. [3 points] What would have happened on a pre-Pascal device (e.g., Kepler) running the same code?

Answer:

c. [3 points] You then increase the allocation to 64 GB on a 16 GB Pascal GPU. Will the `cudaMallocManaged` call succeed? Briefly justify.

Answer:

d. [4 points] Name the API used to give the runtime a hint that data will be **read-mostly**, and describe what the runtime does as a result.

Answer:

5. **[12 points] Tiled SGEMM Analysis**
Consider a square matrix multiply with `Width = 1024`.

   a. [3 points] For `TILE_WIDTH = 16`, how many global memory loads does each *thread block* perform per phase?

   Answer: [ ]

   b. [3 points] For `TILE_WIDTH = 16`, how many phases (outer-loop iterations) does each block execute?

   Answer: [ ]

   c. [3 points] For `TILE_WIDTH = 32`, how many bytes of shared memory does each block consume?

   Answer: [ ]

   d. [3 points] On an SM with 48 KB of shared memory, how many `TILE_WIDTH = 32` blocks can be co-resident based on the shared-memory limit alone?

   Answer: [ ]

Multiple Choice. Please completely fill in the circle ◯ for your answer. **[12 points total]**

6. [3 points] Which CUDA API call *actually moves data* between processors in a Unified Memory program (i.e., does not merely hint at future behavior)?

   ◯ `cudaMemAdvise(..., cudaMemAdviseSetReadMostly, ...)`

   ◯ `cudaMemAdvise(..., cudaMemAdviseSetPreferredLocation, ...)`

   ◯ `cudaMemAdvise(..., cudaMemAdviseSetAccessedBy, ...)`

   ◯ `cudaMemPrefetchAsync(...)`

7. [3 points] An access pattern `A[expr]` in a CUDA kernel is best classified as **coalesced** when:

   ◯ `expr` is a multiple of `threadIdx.x`

   ◯ `expr` = (something independent of `threadIdx.x`) + `threadIdx.x`

   ◯ `expr` = `threadIdx.y * Width + threadIdx.x * Width`

   ◯ `expr` = a function of `blockIdx.x` only

8. [3 points] On a Kepler GPU using Hyper-Q with **32 hardware work queues**, two processes share the GPU via MPS, and each process issues 5 streams. What is the most likely performance effect for this configuration?

   ◯ Full 32-way concurrency, no false dependencies

   ◯ False dependencies/serialization due to more streams per client than channels

   ◯ MPS automatically rebalances work across GPUs to avoid contention

   ◯ Kernels from different processes always serialize, regardless of MPS

9. [3 points] In a Dynamic Parallelism kernel, if every thread in a 256-thread block executes the line `child<<<1,1>>>(...)` without any guard, how many child kernel invocations are spawned by that block?

   ◯ 1

   ◯ 32 (one per warp)

   ◯ 256

   ◯ 0 — only the host can launch kernels

True or False. For the following questions, fill the answer box with (T) True or (F) False. **[12 points total]**

10. `atomicAdd` returns the new value (after the add). [ ]

11. Memory coalescing rules apply to shared-memory accesses the same way they do to global memory. [ ]

12. In the tiled SGEMM kernel, the two `__syncthreads()` calls per phase can be safely removed for large enough block sizes. [ ]

13. `cudaDeviceSynchronize()` called inside a device kernel implies `__syncthreads()` as well. [ ]

14. On a dual-IOH server, peer-to-peer (P2P) copy between two GPUs attached to different IOHs can still complete correctly. [ ]

15. `cudaMemAdviseSetAccessedBy` causes the data to immediately move to the indicated device. [ ]

16. [6 points] You write a tiled matrix-multiply with `TILE_WIDTH = 16` for a **non-square** input: M is $j \times k$, N is $k \times l$, P is $j \times l$, with $j = 1000$, $k = 800$, $l = 1500$.

    a. [2 points] What are the x and y dimensions of your grid?

    Answer: [ ]

    b. [2 points] Write the boundary condition a thread must check before loading its element of the **M tile** (use variables `Row, p, TILE_WIDTH, tx, j, k`).

    Answer: [ ]

    c. [2 points] Write the boundary condition a thread must check before **writing** its `P` element (use `Row, Col, j, l`).

    Answer: [ ]

17. [12 points] **Dynamic Parallelism trace.**
Consider the following device-side code:

```
__global__ void parent(int *out) {
    int tid = threadIdx.x;              // PC = A
    if (tid < 4) {                      // block of 32 threads
        child<<<1, 8>>>(out, tid);      // PC = B
    }
    cudaDeviceSynchronize();            // PC = C
    out[tid] += 1;                      // PC = D
}
```

a. [3 points] How many *child grids* are spawned by this block? Briefly justify.

Answer:

b. [3 points] At PC = C, exactly which launches have finished from the perspective of thread `tid = 5`, which never executed the launch line?

Answer:

c. [3 points] At PC = D, can thread `tid = 5` safely read `out[k]` written by a child launched by thread 0? Why or why not?

Answer:

d. [3 points] If you replaced `cudaDeviceSynchronize()` with `__syncthreads()`, would the program still be correct? Briefly justify.

Answer:

18. [4 points] In a histogram kernel, two threads in the *same* block simultaneously attempt `atomicAdd(&histo_private[3], 1)` where `histo_private` lives in shared memory and currently holds 7. After both atomics complete, what is the value of `histo_private[3]`, and what values did the two threads receive as return values?

Answer: